



OPERATING SYSTEM

Assignment#01

Abdul Mateen Butt
BSEF23M036

Assignment# 01

Part A: Preemptive Round Robin

(1): *Gantt Chart*

P1	P1	Idle	P2	P3	P2	P3	P2	P4	P3	P4	Idle	P5	P6	
0	10	20	25	35	45	55	65	70	80	85	90	100	110	120

2: *Turnaround Time*

Process TAT

Formula: $TAT = CT - AT$

$$P1 = 20 - 0 = 20$$

$$P2 = 70 - 25 = 45$$

$$P3 = 85 - 30 = 55$$

$$P4 = 90 - 60 = 30$$

$$P5 = 110 - 100 = 10$$

$$P6 = 120 - 105 = 15$$

P1 20

P2 45

P3 55

P4 30

P5 10

P6 15

3: *Waiting Times*

Process WT

$$P1 = 20 - 20 = 0$$

$$P2 = 45 - 25 = 20$$

$$P3 = 55 - 25 = 30$$

$$P4 = 30 - 15 = 15$$

$$P5 = 10 - 10 = 0$$

$$P6 = 15 - 10 = 5$$

P1 0

P2 20

P3 30

P4 15

P5 0

P6 5

4) Average Turnaround Time = sum of all TAT/total no of TAT values = 29.17

5) Average Waiting Time = sum of all waiting Time/total no of WT values = 11.67

Part B: Preemptive Priority Scheduling

P1	Idle	P2	P3	P4	P3	Idle	P5	P6	P5	
0	20	25	50	60	75	90	100	105	115	120

1: Turnaround Time

Process CT AT TAT

P1 20 0 20

P2 50 25 25

P3 90 30 60

P4 75 60 15

P5 120 100 20

P6 115 105 10

2: Waiting Times:

Process TAT BT WT

P1 20 20 0

P2 25 25 0

P3 60 25 35

P4 15 15 0

P5 20 10 10

P6 10 10 0

3:CPU Utilization

Formula: $CPU\ Utilization = \frac{Busy\ Time}{Total\ Time} \times 100$

Total burst time: $20 + 25 + 25 + 15 + 10 + 10 = 105$

Total elapsed time: $120 - 0 = 120$ **CPU Utilization:** $\frac{105}{120} \times 100 = 87.5\%$

Average Turnaround Time = 25 **Average Waiting Time** = 7.5

Comparison

Metric	Round Robin	Priority
Average Turnaround Time	29.17	25
Average Waiting Time	11.67	7.5

Question02:

```
#include <iostream>
#include <fstream>
#include <queue>
#include <vector>
#include <iomanip>
using namespace std;
struct Process {
    string pid;
    int arrival;
    int burst;
    int remaining;
    int completion;
    int waiting;
    int turnaround;
    int response;
    bool started = false;
```

```

};

int main()
{
    ifstream fin("input.txt");
    ofstream fout("output.txt");

    if (!fin) {
        cout << "Error opening input file" << endl;
        return 0;
    }

    int n, quantum;

    fin >> n;
    fin >> quantum;

    vector<Process> p(n);

    for (int i = 0; i < n; i++) {
        fin >> p[i].pid >> p[i].arrival >> p[i].burst;
        p[i].remaining = p[i].burst;
        p[i].response = -1;
    }

    queue<int> q;
    vector<bool> inQueue(n, false);

    int currentTime = 0;
    int completed = 0;

```

```

vector<pair<string, int>> gantt;

for (int i = 0; i < n; i++) {
    if (p[i].arrival <= currentTime) {
        q.push(i);
        inQueue[i] = true;
    }
}

while (completed < n) {
    if (q.empty()) {
        currentTime++;

        for (int i = 0; i < n; i++) {
            if (!inQueue[i] && p[i].arrival <= currentTime && p[i].remaining > 0) {
                q.push(i);
                inQueue[i] = true;
            }
        }
        continue;
    }
    int i = q.front();
    q.pop();
    if (!p[i].started) {
        p[i].response = currentTime - p[i].arrival;
        p[i].started = true;
    }
    int executeTime = min(quantum, p[i].remaining);

```

```

gantt.push_back({p[i].pid, currentTime});
currentTime += executeTime;
p[i].remaining -= executeTime;
for (int j = 0; j < n; j++) {
    if (!inQueue[j] && p[j].arrival <= currentTime && p[j].remaining > 0) {
        q.push(j);
        inQueue[j] = true;
    }
}
if (p[i].remaining > 0) {
    q.push(i);
}
else {
    completed++;
    p[i].completion = currentTime;
    p[i].turnaround = p[i].completion - p[i].arrival;
    p[i].waiting = p[i].turnaround - p[i].burst;
}
}

cout << "\nGantt Chart:\n";
for (auto &g : gantt) {
    cout << "| " << g.first << " ";
}
cout << "\n";
cout << "\nProcess Details:\n";
cout << left << setw(10) << "PID"
    << setw(10) << "AT"
    << setw(10) << "BT"

```

```
<< setw(15) << "WT"
<< setw(15) << "RT"
<< setw(15) << "TAT" << endl;
```

```
fout << left << setw(10) << "PID"
    << setw(10) << "AT"
    << setw(10) << "BT"
    << setw(15) << "WT"
    << setw(15) << "RT"
    << setw(15) << "TAT" << endl;
```

```
double totalWT = 0, totalRT = 0, totalTAT = 0;
```

```
for (int i = 0; i < n; i++) {
    cout << left << setw(10) << p[i].pid
        << setw(10) << p[i].arrival
        << setw(10) << p[i].burst
        << setw(15) << p[i].waiting
        << setw(15) << p[i].response
        << setw(15) << p[i].turnaround << endl;
```

```
fout << left << setw(10) << p[i].pid
    << setw(10) << p[i].arrival
    << setw(10) << p[i].burst
    << setw(15) << p[i].waiting
    << setw(15) << p[i].response
    << setw(15) << p[i].turnaround << endl;
```

```
totalWT += p[i].waiting;
```



```
    totalRT += p[i].response;
    totalTAT += p[i].turnaround;
}
cout << "\nAverage Waiting Time: " << totalWT / n << endl;
cout << "Average Response Time: " << totalRT / n << endl;
cout << "Average Turnaround Time: " << totalTAT / n << endl;
fout.close();
fin.close();
return 0;
}
```